PPI Tracker
Preventing cell editing conflicts

A practical implementation guide | 4 November 2026

Purpose: Detect when a cell has changed while a user is editing it, and warn the user before their save overwrites newer data.

1. What the first requirement means

Two people may edit the same cell at the same time. Without a check, whoever saves last can silently replace the other person's work. The requested feature detects this situation at save time.

Step	Example
1	The Status cell contains Pending. User A starts editing it.
2	User B changes the same cell to Approved and saves.
3	User A tries to save Rejected.
4	The application detects the intervening change and displays a warning.

Where the check must run

Direct cell editing: Check when the user completes the edit and the app attempts to save it.
Bulk editing dialog: Check the cells being changed when the user clicks Save.

Requirement versus recommended behavior

The attachment requests conflict detection and a warning. It does not specify whether users may overwrite a newer value, skip conflicting cells, or cancel. This guide recommends stopping the conflicting save, preserving the draft, and letting the user review the latest value.

The recommended implementation uses **optimistic concurrency control**: users edit normally, and the server verifies that the data they started with is still current before accepting their save.

2. How revisions detect a conflict

Each cell has a value and an internal revision number. Every successful change to that cell increases its revision. Users do not need to see revisions in the table.

```
Row:       Shot_010
Column:    Status
Value:     Pending
Revision:  5
```

A. Capture the starting state

When User A starts direct editing, retain an immutable snapshot of the value and revision shown in the editor. When the bulk dialog opens, retain a separate snapshot for each target cell. If data was already loaded earlier, keep the revision associated with the displayed value; do not pair an old value with a freshly fetched revision.

```
Original value:    Pending
Original revision: 5
Draft value:       Rejected
```

B. Another user saves

User B saves Approved. The server writes the new value and increases this cell's revision from 5 to 6. User A's draft remains Rejected, and their original revision remains 5.

C. Send a conditional save

User A's request means: **Save Rejected only if this cell is still at revision 5.** The server compares expected revision 5 with stored revision 6 and rejects the write because they differ.

Expected revision	Stored revision	Server action
5	5	Save the value and increment the revision.
5	6	Reject the conflicting save; keep the stored value.

Why use cell-level revisions?

A single revision for the whole row would also flag edits to unrelated columns. Per-cell revisions allow one user to change Status while another changes Notes in the same row. Revisions also detect a value changing away and then back again, which comparing only the text would miss.

3. API changes and endpoint choices

A new endpoint is not required for single-entity conflict detection. Extend the existing update endpoint already called by this application:

```
PUT /api/tracker/entities/{entityID}
```

Add expected revisions for every field being changed. The following illustrates the new fields; keep other existing request metadata such as root, group, user, and studio as required.

```
{
  "data": {"status": "Rejected"},
  "expected_revisions": {"status": 5}
}
```

Successful response

After saving, return the accepted values and new revisions so the app can refresh its local state. This response shape is a proposal, not a verified backend contract.

```
{
  "data": {"status": "Rejected"},
  "revisions": {"status": 6}
}
```

Conflict response

Return HTTP **409 Conflict** with the current values and revisions of the conflicting cells. Return editor identity and update time only if that metadata is recorded and available.

```
{
  "error": "cell_conflict",
  "conflicts": [{
    "field": "status",
    "current_value": "Approved",
    "current_revision": 6
  }]
}
```

The entity-loading API must also return revisions alongside values. A revision is a concurrency token, not proof of who changed a cell; identifying the editor requires separate metadata.

When a new endpoint is useful

For bulk editing with **save all or save nothing** behavior, add a batch operation, for example `POST /api/tracker/entities/batch-update`. A new batch endpoint is a design choice; an existing endpoint could instead be extended to accept a batch. The essential requirement is one server-side transaction for the entire batch.

4. Server-side safety and bulk saves

The comparison and write must be atomic

A desktop-only check has a race condition: the app checks revision 5, another user saves revision 6, and then the app writes its value. The server must compare the revision and perform the write as one indivisible operation.

For a relational design with a record per cell, the logic could look like this:

```
UPDATE cells
SET value = :new_value,
    revision = revision + 1
WHERE entity_id = :entity_id
  AND field_key = :field_key
  AND revision = :expected_revision;
```

One affected record means success. Zero affected records means the expected state was not found; the server should distinguish a changed cell from a deleted or missing cell. This SQL is illustrative. The actual storage schema and database technology have not been inspected.

Bulk example

Target cell	Expected	Current	Check
Shot_010 / Status	5	5	Pass
Shot_020 / Status	3	4	Conflict
Shot_030 / Status	8	8	Pass

Recommended behavior: reject the entire batch, return the conflict for Shot_020, and keep every draft value available for review. Saving unaffected cells is another possible product choice, but it must be made explicit to users.

Transaction sequence

1. Receive all target cells, draft values, and expected revisions.
2. Begin a database transaction.
3. Validate permissions and conditionally update the targeted cells using revision guards.
4. If any guard fails, roll back the whole batch and return conflicts.
5. If all checks succeed, commit and return the resulting values and revisions.

A transaction alone is insufficient if the implementation merely reads revisions and later performs unguarded writes. Use conditional writes or suitable locking/isolation so concurrent saves cannot slip between validation and writing.

Missing entities and connection failures

Protect concurrent creation with a unique entity identity and explicit create semantics. Do not treat a failed fetch as an empty cell. If a network timeout leaves the save outcome unknown, reload/reconcile the server state before presenting a definite outcome or blindly retrying.

5. Changes in your desktop application

The following locations were inspected in the current project. The backend implementation was not inspected. File paths below are relative to the project root.

Code location	Required change
src/ppiTracker/mainWindow.py saveDirectCellEdit()	Capture starting values/revisions when the editor opens. Send expected revisions on completion. Handle conflicts and preserve the draft.
src/ppiTracker/entityEditorDialog.py save()	Capture per-cell snapshots for the dialog. Send expected revisions for targeted cells. Use a batch operation for atomic bulk saving.
src/ppiTracker/apiClient.py updateEntity()	Add expected revisions to the request. Recognize HTTP 409 before raising generic HTTP errors. Return successful values/revisions to the caller.
Backend and entity-loading API	Store and return cell revisions; enforce guarded writes; return structured conflict details.

Current behavior that matters

Both direct editing and the bulk dialog call updateEntity(). That method currently sends new data without expected cell revisions. The direct-edit flow catches failures as generic errors. The bulk dialog loops over rows and saves them individually, so earlier rows can already be saved if a later row fails.

The existing **Confirm Replace Values** message asks whether the user intends to replace an existing value. It does not check whether someone changed that value after editing began. Keep the two decisions conceptually separate.

Recommended conflict dialog

```
Status for Shot_010 changed while you were editing.

Value when you started: Pending
Latest saved value:     Approved
Your unsaved value:     Rejected

[Review latest value]  [Cancel]
```

Keep the draft until the user explicitly discards it or saves successfully. If an intentional overwrite action is offered, send the revision of the latest value the user reviewed. If another change occurs while the warning is open, the next save must detect that conflict again.

For a failed batch, do not update the local table as though the entire operation succeeded. On success, update the local cache with the server's returned values and revisions.

6. Implementation and verification checklist

Suggested implementation order

1. Agree on per-cell revision storage and request/response contracts.
2. Return revisions from entity reads and store them with displayed data.
3. Add atomic revision checks to the existing update endpoint.
4. Extend the API client to send revisions and expose structured conflicts.
5. Add editor snapshots and conflict dialogs to direct editing.
6. Add a transactional batch operation and connect the bulk dialog.
7. Refresh local values/revisions after successful saves and reviewed conflicts.

Important verification cases

Scenario	Expected result
One user edits a cell	Save succeeds and its revision increases.
Two users edit the same cell	First save succeeds; the stale save receives a conflict.
Two users edit different columns in one row	Both saves succeed when the changed cells are independent.
A value changes away and then returns	The stale revision still detects the intervening edits.
Another edit happens while the warning is open	A retry using the reviewed revision conflicts again.
One cell in a bulk batch has changed	With the recommended policy, none of the batch is committed.
Save fails or times out	Draft stays available; uncertain outcomes are reconciled.
A target is deleted or concurrently created	Handle missing/duplicate identity explicitly; do not silently recreate or overwrite.

Decisions to settle before coding

Bulk policy: all-or-nothing is recommended, but partial saves can be supported if clearly reported.
Overwrite option: decide whether users can explicitly replace a reviewed newer value.
Client compatibility: decide how older clients without revision tokens will be handled. Allowing unguarded updates means those clients can still overwrite data.
Other write paths: imports, rollbacks, scripts, and other writers must advance affected cell revisions. Apply expected-revision checks wherever stale overwrites need protection.

Scope of this guide

This document covers the first item in the attached PPI Breakdown: conflict prevention for simultaneous cell editing. It consolidates the explanation and proposed design from this chat. API payloads, SQL, and batch endpoint names are illustrative proposals. No application or backend changes were made while preparing this guide.